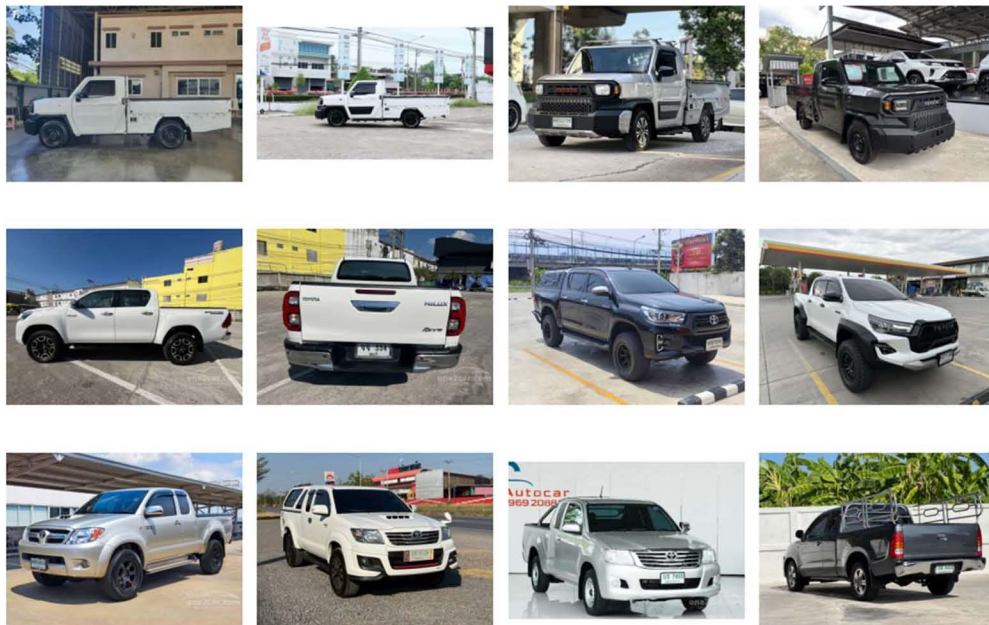


All new DL

1) Dataset description ชุดข้อมูลนี้สร้างขึ้นโดยมุ่งเน้น การจำแนกรถกระบะ Toyota 3 รุ่นที่มีลักษณะใกล้เคียงกัน ได้แก่ Hilux Champ, Revo, และ Vigo

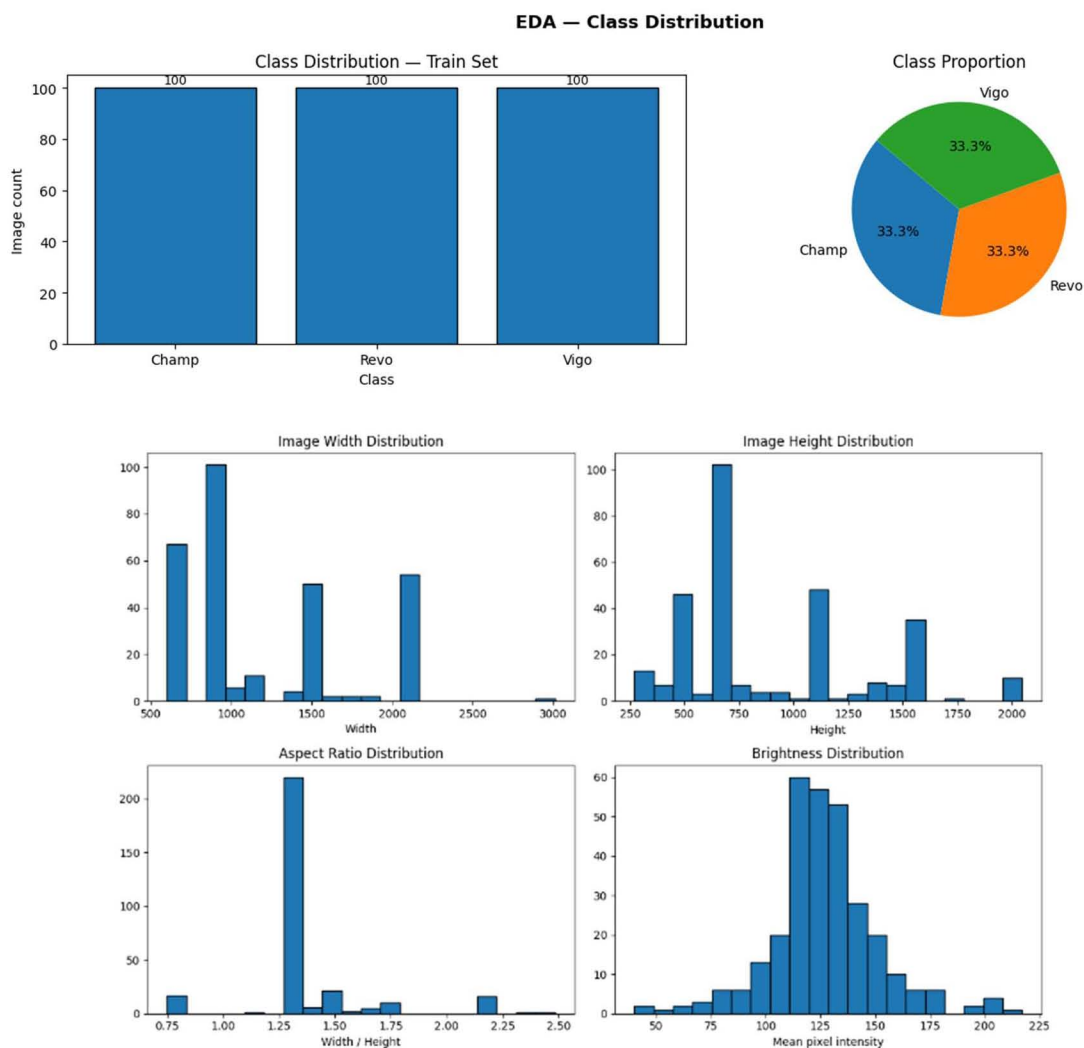
- ที่มาของข้อมูล: ชุดข้อมูลรวบรวมมาจากการสืบค้น (Manual Search) จากแพลตฟอร์มออนไลน์ โดยมีแหล่งที่มาหลัก 2 แหล่ง ได้แก่:
 - Google Images: ค้นหาผ่านคีย์เวิร์ดชื่อรุ่นรถ
 - เว็บไซต์ซื้อขายรถมือสองเช่น <https://www.taladrod.com>, <https://www.toyotasure.com>
 - Facebook: รวบรวมจากกลุ่มซื้อขายรถยนต์มือสอง และแฟนเพจผู้ใช้งานรถกระบะ Toyota (Hilux Champ, Revo, Vigo) เช่น TOYOTA Hilux Champ Thailand TH

Sample Images per Class



- เพื่อให้โมเดลสามารถเรียนรู้ลักษณะของรถได้อย่างสมบูรณ์และลด Bias ได้คัดเลือกรูปภาพให้มีความหลากหลาย เช่น ความหลากหลายของสภาพแวดล้อม: มีทั้งรูปรถหน้าโชว์รูม, รูปรถตามเต็นท์มือสอง, และรูปรถใช้งานจริงบนท้องถนน ความหลากหลายของตัวรถ: กระจายสัดส่วนสีของรถให้คละกัน (ขาว, ดำ, บรอนซ์ ฯลฯ) และเลือกภาพที่มีทั้งมุมมองหน้าตรง มุมเฉียง เพื่อให้เห็นกระจังหน้าและไฟหน้าชัดเจน
- Exploratory Data Analysis (EDA)

- จำนวนภาพ: ทั้งหมด 300 ภาพ โดยมีจำนวนภาพเท่ากันทุกคลาส ได้แก่ Hilux Champ 100 ภาพ, Revo 100 ภาพ และ Vigo 100 ภาพ
- ขนาดและสัดส่วนของภาพ: มีความหลากหลายสูงมาก โดยมีความกว้างตั้งแต่ 600 ถึง 3,012 พิกเซล และมีสัดส่วนกว้างต่อยาว (Aspect Ratio) ตั้งแต่ 0.75 ถึง 2.48
- ความสว่าง: ชุดข้อมูลครอบคลุมรถยนต์ในหลายสภาพแวดล้อม โดยมีตั้งแต่ภาพที่ถ่ายในสภาวะแสงน้อย (ค่าความสว่างต่ำสุดที่ 40.08) ไปจนถึงภาพที่ถ่ายกลางแจ้ง (สูงสุดที่ 217.04)



- จากการทำ EDA พบว่าชุดข้อมูลไม่มีปัญหา Imbalanced Dataset โดยมีจำนวนภาพเท่ากันทุกคลาสจึงสามารถนำเข้าสู่กระบวนการเทรนด้วยวิธีมาตรฐานได้

2) Data preparation

- Data Preprocessing:
 1. `transforms.Resize(...)`: การปรับขนาดรูปภาพเริ่มต้นให้เท่ากัน (ปรับเป็นขนาด 224x224)
 2. `transforms.ToTensor()`: การแปลงข้อมูลรูปภาพให้อยู่ในรูปแบบของ Tensor พร้อมทั้งทำการปรับค่า Pixel values จากเดิม 0-255 ให้อยู่ในช่วง 0.0-1.0
 3. `transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD)`: การทำ Normalization โดยใช้ค่าเฉลี่ย (Mean) และส่วนเบี่ยงเบนมาตรฐาน (Std) ของชุดข้อมูล ImageNet เพื่อให้ข้อมูลมีการกระจายตัวที่เหมาะสมต่อการเรียนรู้ของโมเดล
- Data Augmentation (เฉพาะ Training Set): ใช้เทคนิค Online Data Augmentation
 - `transforms.RandomResizedCrop(224, scale=(0.8, 1.0))`: การสุ่ม Crop ภาพเฉพาะบางส่วนออกมา โดยสุ่มขนาดพื้นที่ตั้งแต่ 80% ถึง 100% ของภาพเดิม แล้วนำมาขยายหรือย่อกลับให้เป็นขนาด 224x224 พิกเซล
 - `transforms.RandomHorizontalFlip(p=0.5)`: การสุ่มพลิกภาพกลับซ้าย-ขวา โดยกำหนดโอกาสที่จะเกิดการพลิกภาพไว้ที่ 50%
 - `transforms.RandomRotation(degrees=15)`: การสุ่มจับภาพเอียงหรือหมุนไปทางซ้ายและขวา ในระดับที่ไม่เกิน 15 องศา
 - `transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1)`: การสุ่มปรับสีและแสงของรูปภาพ
 - `transforms.RandomGrayscale(p=0.1)`: การสุ่มเปลี่ยนภาพเป็นขาวดำ โดยมีโอกาสเกิดขึ้น 10%
- Data Splitting: แบ่งชุดข้อมูลออกเป็น 3 ส่วน ในอัตราส่วนภาพรวมประมาณ 70:20:10 โดยขั้นตอนแรกได้ทำการแยก Test Set ไว้ในโฟลเดอร์ต่างหาก เพื่อใช้เป็นข้อมูลทดสอบขั้นสุดท้าย (Unseen Data) จากนั้นนำข้อมูลส่วนที่เหลือมาแบ่งด้วยเทคนิค Stratified Data Splitting สำหรับ Training Set และ Validation Set ในสัดส่วน 80:20 เพื่อรักษาสัดส่วนของรูปภาพในแต่ละคลาสให้สมดุลและมีการกระจายตัวเท่าเทียมกับชุดข้อมูลตั้งต้น

```
✓ Stratified split complete:
Total train images : 300
Train subset       : 240 (80%)
Val subset         : 60  (20%)
Test (separate dir): 30
Batch size        : 16

Class      Train  Val
Champ      80    20
Revo       80    20
Vigo       80    20
```

- Data Loading: กำหนด Batch Size ไว้ที่ 16 รูปภาพต่อรอบ สำหรับ Training Set ได้มีการใช้การสุ่มสลับลำดับรูปภาพ (Shuffle) ใหม่ทั้งหมดในทุกๆ รอบ Epoch เพื่อป้องกันปัญหาโมเดลจดจำลำดับข้อมูล

3) Model Architecture

- การเลือกโมเดลทั้ง 4 รุ่น มีวัตถุประสงค์เพื่อเปรียบเทียบความคุ้มค่าระหว่าง Accuracy และความเร็วในการประมวลผล สำหรับการนำไปใช้งานจริง โดยครอบคลุมตั้งแต่โมเดลที่กินทรัพยากรสูงสุดไปจนถึงต่ำสุด ช่วยให้ประเมิน Trade-off ได้ชัดเจนว่า งานของเราต้องพึ่งพาเซิร์ฟเวอร์ขนาดใหญ่ หรือสามารถใช้โมเดลเล็กบนอุปกรณ์พกพาได้โดยที่ความแม่นยำยังอยู่ในระดับที่ยอมรับได้

Model	Params	จุดเด่น (Strengths)	ข้อจำกัด (Limitations)
VGG-16	~138.4 ล้าน	โครงสร้างเรียบง่าย สกัดลักษณะเด่น (Feature) พื้นฐานได้ดีเยี่ยม เหมาะเป็น Baseline	ขนาดโมเดลใหญ่มาก กินพื้นที่หน่วยความจำและใช้เวลาในการประมวลผลนานที่สุด
ResNet-50	~25.6 ล้าน	สามารถแทนโมเดลที่มีความลึกมาก ๆ ได้อย่างมีประสิทธิภาพ ให้ความแม่นยำสูง ถือเป็นโมเดลมาตรฐานระดับสากล	โครงสร้างมีความซับซ้อนกว่า VGG เล็กน้อย
DenseNet-121	~8.0 ล้าน	ใช้พารามิเตอร์น้อยแต่ให้ประสิทธิภาพสูงจากการใช้ Feature ซ้ำ	กินพื้นที่หน่วยความจำการ์ดจอ (VRAM) ระหว่างเทรนสูง เนื่องจากต้องเก็บ Feature ของเลเยอร์ก่อนหน้าไว้ทั้งหมด
MobileNet-V3-Large	~5.5 ล้าน	มีน้ำหนักเบาที่สุดและประมวลผลได้รวดเร็ว ออกแบบมาเพื่อใช้งานบนอุปกรณ์พกพา (Mobile/Edge Devices) โดยเฉพาะ	ความสามารถในการสกัดลักษณะเด่นที่ซับซ้อนมาก ๆ อาจเป็นรองโมเดลที่มีขนาดใหญ่กว่า

- Pre-trained CNN: แสดงประสิทธิภาพของโมเดล Pre-trained CNN
 - VGG-16

Pre-trained vgg16 (ImageNet only) — Before Any Fine-tuning
Prediction results on our custom dataset



- ResNet-50

Pre-trained ResNet50 (ImageNet only) — Before Any Fine-tuning
Prediction results on our custom dataset



- DenseNet-121

Pre-trained DenseNet-121 (ImageNet only) — Before Any Fine-tuning
Prediction results on our custom dataset



- MobileNet-V3-Large

Pre-trained MobileNet-V3-L (ImageNet only) — Before Any Fine-tuning
Prediction results on our custom dataset



- Model Architecture Modification:** เก็บโครงสร้าง Convolutional Layers ดั้งเดิมไว้ทั้งหมดเพื่อใช้เป็น Backbone เพื่อทำหน้าที่เป็น Feature Extractor และตัดเฉพาะ Classifier Layer ส่วนปลายดั้งเดิมที่อ้างอิงผลลัพธ์ 1,000 คลาสของ ImageNet ออก เนื่องจากไม่สอดคล้องกับคลาสเป้าหมายในงานนี้ จากนั้นจึงเพิ่ม Fully Connected layer ใหม่เข้าไปแทนที่ เพื่อทำหน้าที่เป็น Custom Classifier สำหรับพยากรณ์คลาสเป้าหมายของเราโดยเฉพาะ

Model	VGG-16	ResNet-50	DenseNet-121	MobileNet-V3-Large
ตำแหน่งที่แทนที่	classifier[6]	fc	classifier	classifier[3]
Layer 1	Dropout(p=0.5)	Dropout(p=0.3)	Dropout(p=0.3)	Dropout(p=0.3)
Layer 2	Linear	Linear	Linear	Linear
in_features	4096	2048	1024	1280
out_features	3	3	3	3

- VGG-16:** คงส่วน features (Backbone) ไว้ทั้งหมด และทำการเขียนทับส่วนหัวที่ตำแหน่ง classifier[6] โดยเพิ่มเข้าไปใหม่ 2 เลเยอร์ ต่อกันตามลำดับ ได้แก่ 1.เลเยอร์ nn.Dropout (พารามิเตอร์ p=0.5) เพื่อช่วยควบคุมปัญหา Overfitting 2.เลเยอร์ nn.Linear รับข้อมูลต่อจาก Dropout โดยมีพารามิเตอร์ in_features=4096 (รับข้อมูลจาก Backbone) และ out_features=3 เพื่อส่งออกผลลัพธ์

- **ResNet-50:** freeze ทุก layer ยกเว้น fc และทำการเขียนทับส่วนหัวที่ตำแหน่ง fc โดยเพิ่มเข้าไปใหม่ 2 เลเยอร์ ต่อกันตามลำดับ ได้แก่ 1. เลเยอร์ nn.Dropout (พารามิเตอร์ $p=0.3$) เพื่อช่วยควบคุมปัญหา Overfitting 2. เลเยอร์ nn.Linear รับข้อมูลต่อจาก Dropout มีพารามิเตอร์ $in_features=2048$ และ $out_features=3$
- **DenseNet-121:** คงส่วน features ไว้ทั้งหมด และทำการเขียนทับส่วนหัวที่ตำแหน่ง classifier โดยเพิ่มเข้าไปใหม่ 2 เลเยอร์ ต่อกันตามลำดับ ได้แก่ 1. เลเยอร์ nn.Dropout (พารามิเตอร์ $p=0.3$) เพื่อช่วยควบคุมปัญหา Overfitting 2. เลเยอร์ nn.Linear รับข้อมูลต่อจาก Dropout มีพารามิเตอร์ $in_features=1024$ และ $out_features=3$
- **MobileNet-V3-Large:** คงส่วน features ไว้ทั้งหมด และทำการเขียนทับเลเยอร์สุดท้ายที่ตำแหน่ง classifier[3] โดยเพิ่มเข้าไปใหม่ 2 เลเยอร์ ต่อกันตามลำดับ ได้แก่ 1. เลเยอร์ nn.Dropout (พารามิเตอร์ $p=0.3$) เพื่อช่วยควบคุมปัญหา Overfitting 2. เลเยอร์ nn.Linear รับข้อมูลต่อจาก Dropout มีพารามิเตอร์ $in_features=1280$ และ $out_features=3$

4) **Training method:** ใช้เทคนิค Two-Stage Fine-Tuning โดยแบ่งการเทรนออกเป็น 2 ช่วง

- Stage 1 Freeze Backbone: `freeze_backbone=True` หมายความว่า weights ของ backbone (ส่วนที่ pretrained มา) ถูก freeze ไว้ทั้งหมด มีเพียง classifier head เท่านั้นที่ `requires_grad=True` และถูก update ในระหว่างเทรน
- Stage 2 Selective Unfreeze: การ unfreeze บางส่วนของ backbone เพื่อให้โมเดลสามารถ fine-tune features ให้เหมาะกับ dataset ของเรามากขึ้น โดยแตกต่างกันตาม architecture
- เทคนิคการทำ hyperparameter tuning: ใช้ W&B Sweep ร่วมกับ Bayesian Optimization ในการค้นหา hyperparameter ที่เหมาะสมที่สุดสำหรับแต่ละ architecture โดยกำหนด search space ของ hyperparameter ที่ต้องการทดลองไว้ดังนี้

Hyperparameter	ค่าที่ทดลอง
Architecture	VGG-16, ResNet-50, DenseNet-121, MobileNet-V3-Large
Stage 1 Epochs	5, 10
Stage 1 Learning Rate	0.0005, 0.001
Stage 2 Epochs	10, 15, 20
Stage 2 Learning Rate	0.00001, 0.00005, 0.0001
Label Smoothing	0.0, 0.05, 0.1

- เนื่องจาก search space ทั้งหมดมีจำนวน combination 432 combinations การทดลองแบบ Grid Search ซึ่งต้องลองทุก combination จึงไม่เหมาะสมเนื่องจากใช้เวลาและทรัพยากรการคำนวณสูงมาก จึงเลือกใช้ Bayesian Optimization โดยให้ W&B Sweep เป็นตัวกำหนดชุด hyperparameter ในแต่ละ run

โดยอัตโนมัติ ซึ่ง Bayesian Optimization จะเรียนรู้จากผลลัพธ์ของ run ก่อนหน้าและนำมาประมาณค่าว่า hyperparameter ชุดใดน่าจะให้ผลดีในการ run ถัดไป ทำให้สามารถค้นหา hyperparameter ที่ดีได้อย่างมีประสิทธิภาพโดยไม่จำเป็นต้องลองทุก combination

- Metric ที่ใช้เป็นเกณฑ์ในการคัดเลือกคือ Validation F1 Score (Weighted) โดยกำหนดเป้าหมายเป็น maximize เมื่อเสร็จสิ้นกระบวนการ Sweep แล้วจึงนำชุด hyperparameter ที่ให้ค่า Validation F1 สูงสุดของแต่ละ architecture มาใช้ในการเทรนโมเดลจริง ดังแสดงในตารางด้านล่าง

Parameters	Stage 1 (Freeze Backbone)				Stage 2 (Unfreeze)			
	VGG-16	ResNet-50	DenseNet-121	MobileNet-V3-Large	VGG-16	ResNet-50	DenseNet-121	MobileNet-V3-Large
Loss	CrossEntropyLoss				CrossEntropyLoss			
Label Smoothing	0.05	0.0	0.0	0.0	0.05	0.0	0.0	0.0
Optimizer	Adam				AdamW	Adam	Adam	Adam
Learning Rate	0.001	0.001	0.001	0.001	0.0001	0.0001	0.0001	0.00005
LR Scheduler	CosineAnnealing				CosineAnnealing			
Max Epochs	10	10	5	5	20	10	10	20
Early Stopping Patience	5	5	5	5	5	5	5	5
Early Stopping Min Delta	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001	0.0001
Layers Updated	Classifier head only				features[24:] + classifier	layer4 + fc	denseblock4 + norm5 + classifier	features[-3:] + classifier

5) Evaluation metrics & Error Analysis

- Learning Curves
- การประเมินภาพรวมทุกคลาส: Accuracy, Precision, Recall และ F1-Score
- การประเมินแยกรายคลาส: Precision, Recall และ F1-Score
- Error Analysis: Confusion Matrix, GradCAM

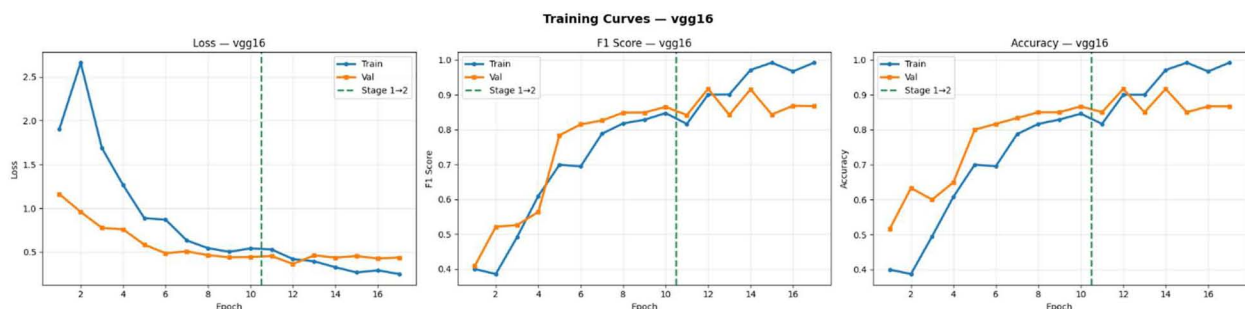
คำนวณมาแบบ macro หรือ micro หรือ weighted?

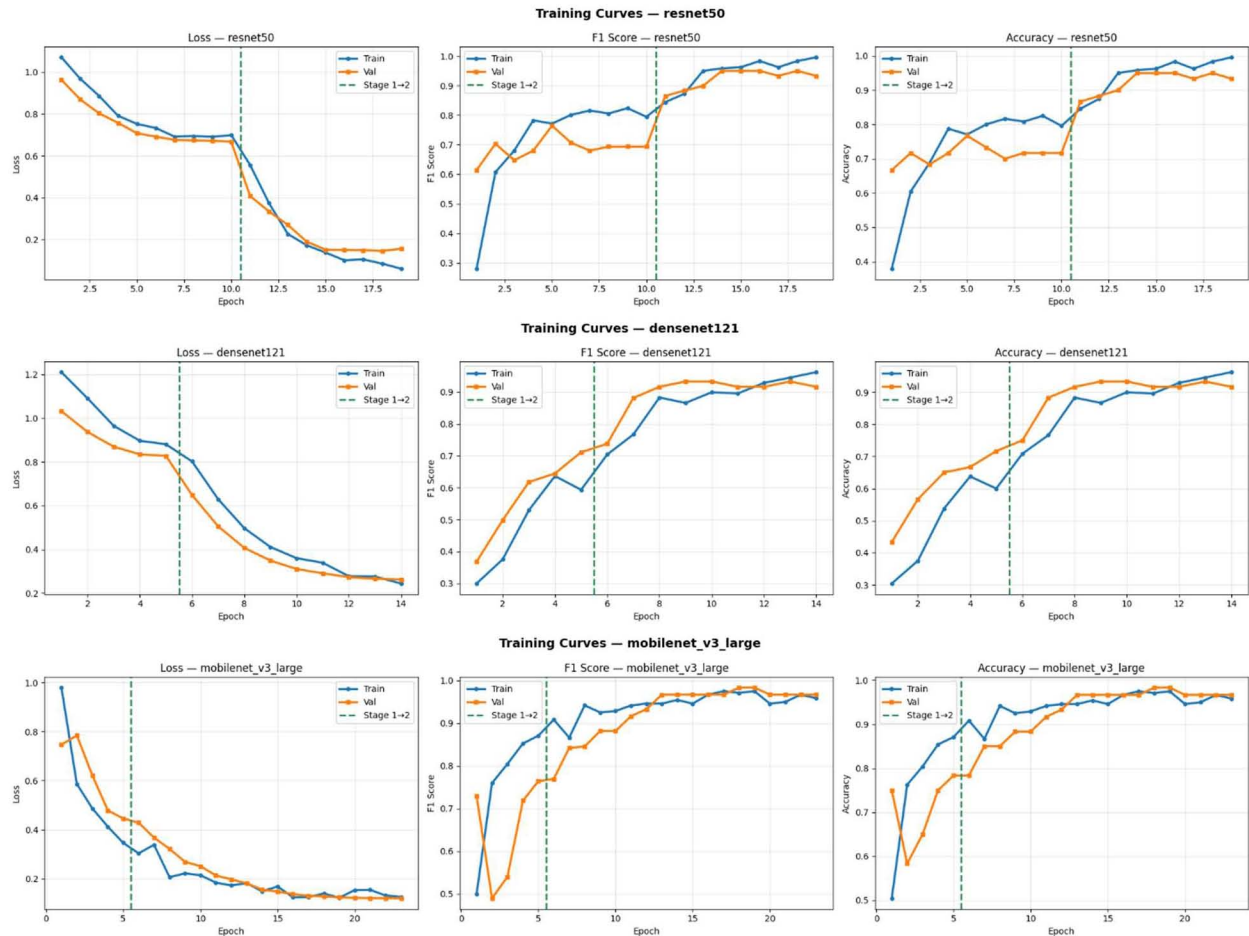
เลือกตัวไหนมาโชว์ (best seed, median seed)? หรือสุ่มมา?

6) Experimental results

■ Learning Curves

- **VGG-16:** พบว่าในช่วง Stage 1 ค่า Train Loss เริ่มต้นสูงและลดลงอย่างรวดเร็ว ขณะที่ Val Loss เริ่มต้นที่ประมาณ 1.0 และลดลงอย่างสม่ำเสมอ โดย F1 Score และ Accuracy ของทั้ง Train และ Val ค่อย ๆ สูงขึ้นพร้อมกัน เมื่อเข้าสู่ช่วง Stage 2 พบว่า Train F1 และ Accuracy ยังคงสูงขึ้นต่อเนื่อง แต่ Val F1 และ Accuracy เริ่มหยุดนิ่งและมีความผันผวน ส่งผลให้ช่องว่างระหว่าง Train และ Val กว้างขึ้นอย่างชัดเจน ซึ่งบ่งชี้ว่าโมเดลมีแนวโน้ม Overfitting อันเป็นผลมาจากจำนวน Parameter ที่สูงของ VGG-16 ประกอบกับขนาดของ Dataset ที่มีอยู่อย่างจำกัด
- **ResNet-50:** พบว่าในช่วง Stage 1 ค่า Train และ Val Loss ลดลงพร้อมกันอย่างสม่ำเสมอ แต่ Val F1 Score ค่อนข้างคงที่ ในขณะที่ Train F1 สูงขึ้นเรื่อยๆ แสดงให้เห็นว่า Classifier Head ยังเรียนรู้ feature ที่เป็นประโยชน์ได้ไม่เพียงพอในขณะที่ Backbone ยังถูก Freeze อยู่ เมื่อเข้าสู่ช่วง Stage 2 หลังจากการ Unfreeze layer4 พบว่า Val F1 กระโดดขึ้นอย่างชัดเจนและทั้ง Train กับ Val เข้าใกล้กัน ซึ่งแสดงให้เห็นว่าการ Fine-tune high-level features ใน layer4 มีประสิทธิภาพสูง โดยไม่มีสัญญาณของ Overfitting
- **DenseNet-121:** พบว่าในช่วง Stage 1 มีเพียง 5 Epochs Train Loss ลดลงเร็วและต่อเนื่อง แต่ Val Loss ลดลงช้ากว่าและมีระยะห่างจาก Train Loss พอสมควร ส่งผลให้ Val F1 อยู่ในระดับที่ค่อนข้างต่ำ ซึ่งบ่งชี้ว่า Stage 1 ที่สั้นเกินไปทำให้ Classifier Head ยังไม่ Stable เพียงพอ เมื่อเข้าสู่ช่วง Stage 2 หลังจากการ Unfreeze denseblock4 และ norm5 พบว่า Val F1 และ Accuracy กระโดดขึ้นอย่างชัดเจนและไล่ตาม Train ได้ดี แสดงให้เห็นว่าการ Fine-tune high-level features มีประสิทธิภาพสูง โดยไม่มีสัญญาณของ Overfitting
- **MobileNet-V3-Large:** พบว่าในช่วง Stage 1 Train Loss ลดลงอย่างรวดเร็วตั้งแต่ต้น ในขณะที่ Val Loss และ Val F1 มีความผันผวนสูงมาก ซึ่งบ่งชี้ว่า Classifier Head ยังเรียนรู้ไม่เสถียรในช่วง Stage 1 ที่มีเพียง 5 Epochs เมื่อเข้าสู่ช่วง Stage 2 หลังจากการ Unfreeze features[-3:] พบว่าทั้ง Train และ Val มีเสถียรภาพเพิ่มขึ้นอย่างชัดเจน โดย F1 และ Accuracy สูงขึ้นต่อเนื่อง และไม่มีสัญญาณของ Overfitting





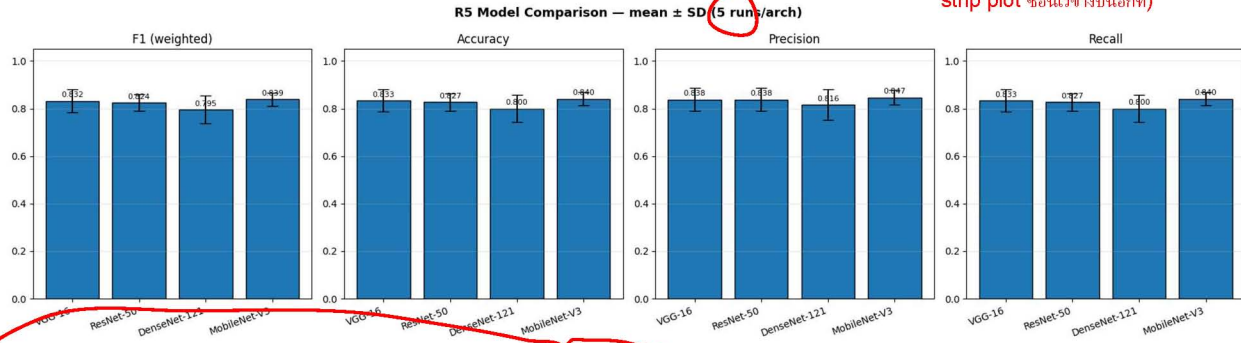
- การประเมินภาพรวมทุกคลาส: Accuracy, Precision, Recall และ F1-Score จากการทดลองพบว่าโมเดลมี F1-Score เรียงลำดับดังนี้:

- MobileNet-V3-Large ทำได้ดีที่สุดด้วย F1 (weighted) = 0.8386 และมี SD ต่ำที่สุดที่ 0.0275 บ่งชี้ว่ามีทั้งประสิทธิภาพและความเสถียรสูงที่สุดในกลุ่ม
- VGG-16 ได้คะแนนรองลงมาที่ 0.8322 แต่มี SD สูงกว่าที่ 0.0473 แสดงว่าผลลัพธ์มีความผันผวนระหว่างรอบการทดสอบมากกว่า
- ResNet-50 อยู่ที่ 0.8245 ด้วย SD 0.0354 ถือว่าอยู่ในระดับกลางทั้งประสิทธิภาพและความเสถียร
- DenseNet-121 มี F1-score ต่ำที่สุดที่ 0.7947 และ SD 0.0591 ทำให้เป็นโมเดลที่น่ากังวลที่สุด ในแง่ทั้งประสิทธิภาพและความเสถียร

Model	F1 (weighted)	Accuracy	Precision	Recall
VGG-16	0.8322 ± 0.0473	0.8333 ± 0.0471	0.8380 ± 0.0481	0.8333 ± 0.0471
ResNet-50	0.8245 ± 0.0354	0.8267 ± 0.0365	0.8382 ± 0.0484	0.8267 ± 0.0365
DenseNet-121	0.7947 ± 0.0591	0.8000 ± 0.0577	0.8156 ± 0.0637	0.8000 ± 0.0577
MobileNet-V3-Large	0.8386 ± 0.0275	0.8400 ± 0.0279	0.8470 ± 0.0313	0.8400 ± 0.0279

ผลนี้คือบน train หรือ val หรือ test set?

กรณีนี้เนื่องจากความต่างของแต่ละโมเดลไม่มาก ทำให้กราฟแทบไม่ต่อล้อ
 นัยยะเพิ่มเติมจากตารางสรุป **mean $\pm$ SD** ในหน้าก่อน
 - กรณีทดลองน้อยกว่า 10 รอบ อาจใช้ผลของทุกรอบเป็น **Dot/Strip plot** แทน จะเห็นจุดข้อมูลและข้อมูลไม่ซ้ำซ้อนกับตาราง
 - แต่ถ้าทดลองเยอะ ≥ 10 รอบ อาจใช้แสดงเป็น **box plot** (และใช้ **strip plot** ซ้อนไว้ข้างบนอีกที)



วิธีทดสอบทางสถิติมีหลายวิธี อาจต้องดูหลาย ๆ ประเด็นเพื่อเลือกวิธีที่เหมาะสม ตั้งแต่รันซ้ำกี่รอบ, จะทดสอบโมเดลเป็นคู่ (2 ตัว) หรือมากกว่านั้น, แต่ละโมเดลมีการใช้ **seed** แต่ละรอบเหมือนกันทุกโมเดลไหม (เช่น ทุกโมเดลใช้ **seed 5** รอบเหมือนกันหมดคือ [1,22,33,44,55]) หรือแต่ละโมเดลใช้ **random seed** ต่างกันทุกรอบอิสระกันหมด

- จากการทดสอบนัยสำคัญทางสถิติ (**Welch's t-test**) ผลการเปรียบเทียบทุกคู่พบว่า ไม่มีความแตกต่างอย่างมีนัยสำคัญทางสถิติในทุกคู่ ($p > 0.05$) แม้ MobileNet-V3-Large จะมี F1-score สูงที่สุด และ DenseNet-121 ต่ำที่สุด แต่ความแตกต่างดังกล่าวไม่เพียงพอที่จะสรุปได้ว่าโมเดลใดดีกว่ากันอย่างแท้จริง

Model Pair		Mean F-1	Mean F-1	p-value	Significant ($p < 0.05$)
(1)	(2)	(1)	(2)		
VGG-16	ResNet-50	0.8322	0.8245	0.7772	No
VGG-16	DenseNet-121	0.8322	0.7947	0.3006	No
VGG-16	MobileNet-V3-Large	0.8322	0.8386	0.8025	No
ResNet-50	DenseNet-121	0.8245	0.7947	0.3673	No
ResNet-50	MobileNet-V3-Large	0.8245	0.8386	0.5022	No
DenseNet-121	MobileNet-V3-Large	0.7947	0.8386	0.1852	No

- การประเมินแยกแยะคลาส: Precision, Recall และ F1-Score

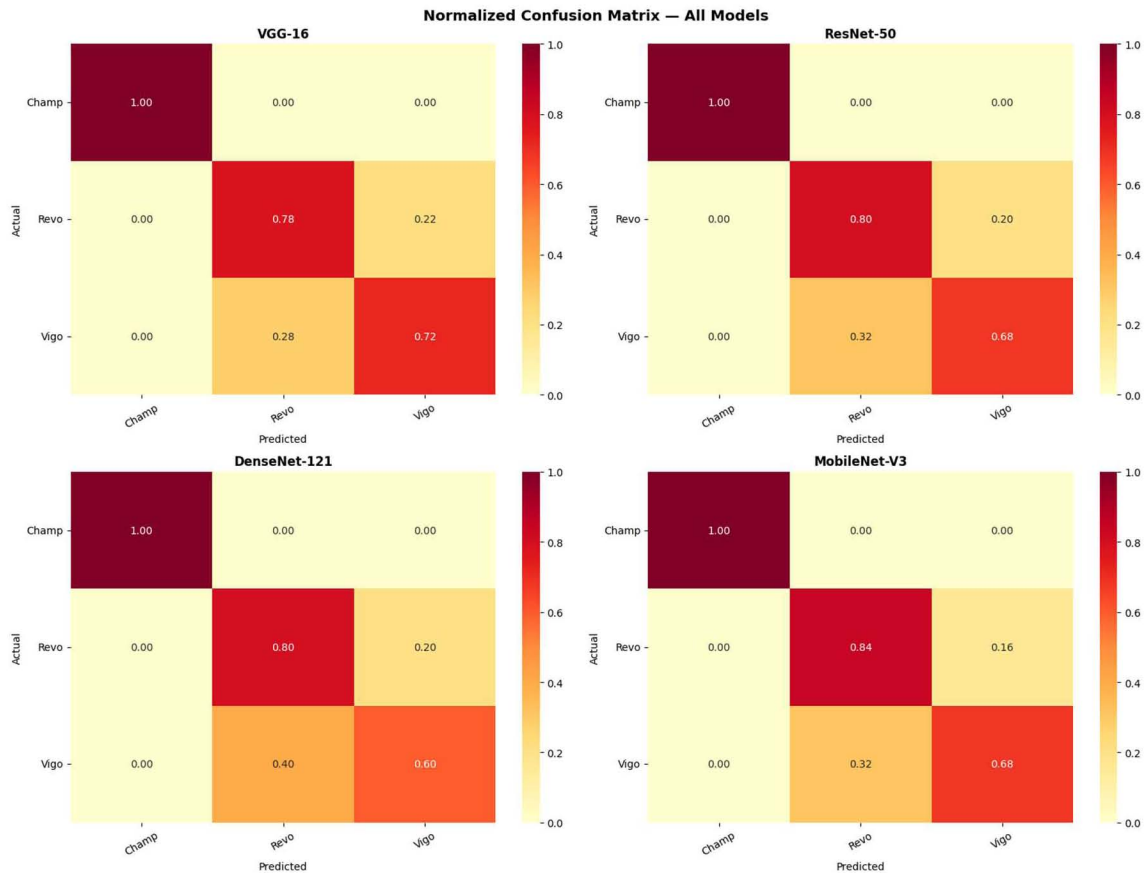
ปกติรายงานค่า **p** ก็เพียงพอ แต่ถ้าอยากวิเคราะห์เพิ่มอีก สามารถเพิ่ม **"effect size"** เข้ามาอีกคอลัมน์ได้

- Champ** คือคลาสที่โมเดลจำแนกได้ดีที่สุด ทุกโมเดลจำแนกได้ถูกต้องสมบูรณ์ที่ 1.0000 โดยไม่มีความผิดพลาดเลย
 - Revo** อยู่ในระดับกลาง F1-score อยู่ที่ 0.66–0.74 โดย VGG-16 ทำได้ดีที่สุดที่ 0.7445 และ DenseNet-121 ต่ำที่สุดที่ 0.6582 ซึ่งยังมี SD สูงถึง 0.0530 บ่งชี้ถึงความไม่เสถียร
 - Vigo** มีผลลัพธ์หลากหลายกว่าคลาสอื่น โดย MobileNet-V3-Large ทำได้ดีที่สุดที่ 0.8170 ตามด้วย ResNet-50 ที่ 0.7983 ขณะที่ DenseNet-121 และ VGG-16 ทำได้ต่ำกว่าที่ 0.7886 และ 0.7694 ตามลำดับที่น่าสังเกตคือ Vigo มี SD สูงในหลายโมเดล โดยเฉพาะ DenseNet-121 ที่ 0.1402 สะท้อนถึงความผันผวนของผลลัพธ์ระหว่างการทดสอบ

Model	Class	F1-score	Precision	Recall
VGG-16	Champ	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
	Revo	0.7445 ± 0.0957	0.7800 ± 0.0837	0.7580 ± 0.0642
	Vigo	0.7694 ± 0.0750	0.7200 ± 0.1304	0.7387 ± 0.0823
ResNet-50	Champ	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
	Revo	0.7162 ± 0.0354	0.8000 ± 0.1581	0.7491 ± 0.0734
	Vigo	0.7983 ± 0.1377	0.6800 ± 0.0837	0.7243 ± 0.0431
DenseNet-121	Champ	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
	Revo	0.6582 ± 0.0530	0.8000 ± 0.2236	0.7137 ± 0.1366
	Vigo	0.7886 ± 0.1402	0.6000 ± 0.0707	0.6703 ± 0.0491
MobileNet-V3-Large	Champ	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000
	Revo	0.7239 ± 0.0271	0.8400 ± 0.0894	0.7762 ± 0.0489
	Vigo	0.8170 ± 0.0768	0.6800 ± 0.0447	0.7397 ± 0.0375

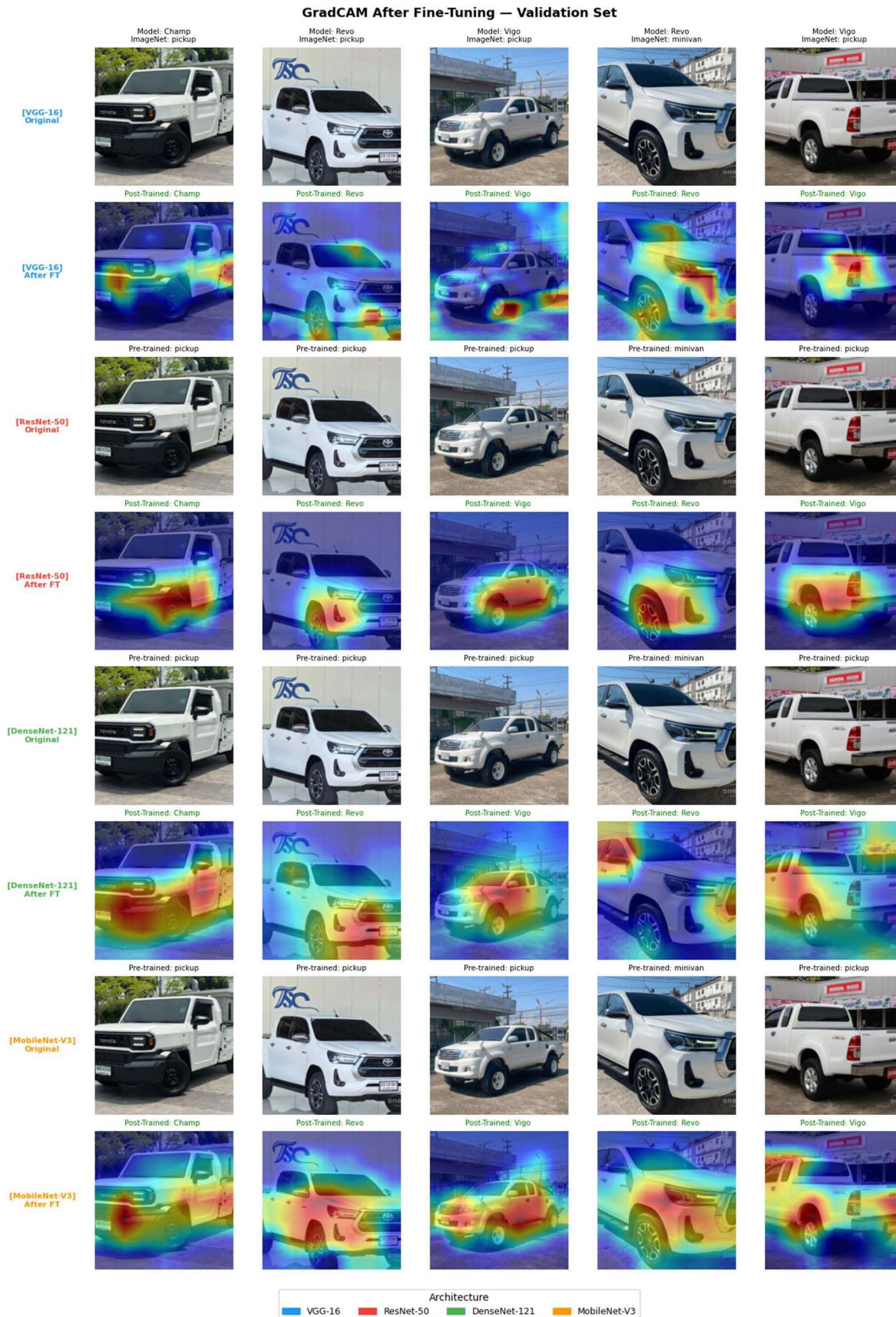
- Confusion Matrix

- **Champ** ทุกโมเดลจำแนกได้ถูกต้อง 1.00 โดยไม่มีการสับสนกับคลาสอื่นเลย
- **Revo** โมเดลส่วนใหญ่จำแนกได้ค่อนข้างดี โดย MobileNet-V3 ทำได้ดีที่สุดที่ 0.84 ตามด้วย ResNet-50 และ DenseNet-121 ที่ 0.80 และ VGG-16 ที่ 0.78 ข้อผิดพลาดที่พบคือโมเดลมักทำนาย Revo เป็น Vigo คิดเป็น 0.16–0.22
- **Vigo** คือคลาสที่มีปัญหามากที่สุด ทุกโมเดลจำแนกได้ต่ำกว่า 0.75 โดย VGG-16 และ MobileNet-V3 ทำได้ 0.72 และ 0.68 ตามลำดับ ขณะที่ DenseNet-121 ต่ำที่สุดที่ 0.60 ที่น่าสังเกตคือทุกโมเดลมักสับสนระหว่าง Vigo กับ Revo สะท้อนว่าทั้งสองคลาสนี้มีลักษณะภาพที่คล้ายคลึงกัน



- GradCAM analysis ใช้เพื่อตรวจสอบว่าโมเดลตัดสินใจจำแนกภาพโดยโฟกัสไปที่บริเวณที่เหมาะสมหรือไม่ โดยสีแดง-เหลืองแสดงพื้นที่ที่โมเดลให้ความสนใจมากที่สุด พบว่าทุกโมเดลสามารถโฟกัสไปที่ตัวรถได้โดยตรง แต่อย่างไรก็ตามแต่ละโมเดลมีลักษณะการโฟกัสที่ต่างกัน
 - VGG-16 แสดง heat map ที่กระจายอย่างไม่สม่ำเสมอและขาดจุดโฟกัสที่ชัดเจน โดยมีแนวโน้มโฟกัสไปที่องค์ประกอบที่ไม่เกี่ยวข้องกับ feature ของรถ เช่น background รอบข้าง
 - ResNet-50 และ MobileNet-V3-Large แสดง heat map ที่มีความเข้มและเฉพาะเจาะจงสูง โดยโฟกัสที่ feature สำคัญของรถเช่น บริเวณกระจกหน้า ไฟหน้า และล้อรถ
 - DenseNet-121 แสดง heat map ที่โฟกัสที่ตัวรถได้ แต่ยังไม่เฉพาะเจาะจงเท่า ResNet-50 และ MobileNet-V3

ถ้าแยก GradCAM กรณีทำนายถูก กับกรณีทำนายผิด
อาจทำให้เห็นภาพที่ปรับเปลี่ยนเพิ่มเติมได้อีก

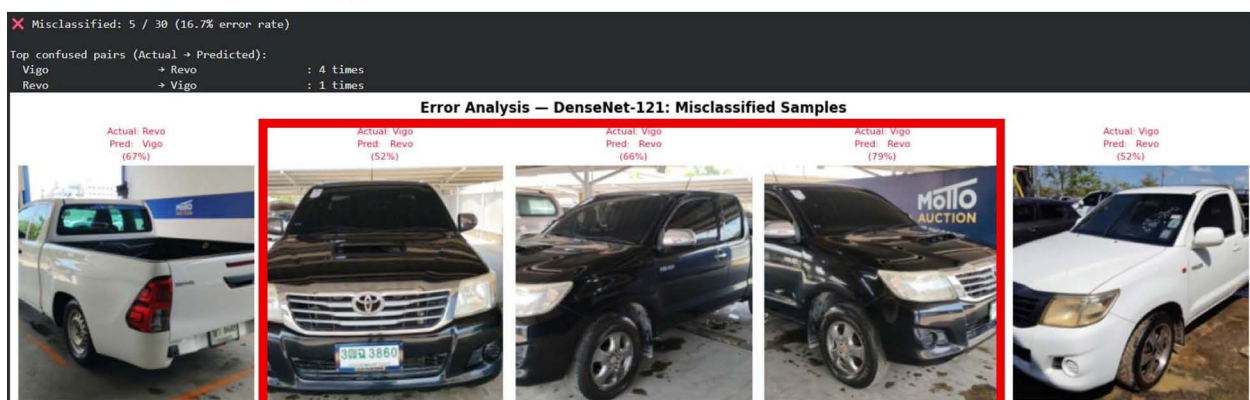


7) Discussion and conclusions

- ผลการทดสอบนัยสำคัญทางสถิติ (Welch's t-test) เมื่อเปรียบเทียบ F1-score ระหว่างโมเดลทุกคู่พบว่า ไม่มีคู่ใดแตกต่างกันอย่างมีนัยสำคัญทางสถิติ ($p > 0.05$ ทุกคู่) แม้ MobileNet-V3-Large จะมี F1-score เฉลี่ยสูงที่สุดที่ 0.8386 แต่เมื่อเทียบกับโมเดลอื่น เช่น VGG-16 ($p = 0.8025$) หรือ ResNet-50 ($p = 0.5022$) ความแตกต่างที่เห็นนั้นอาจเกิดจากความแปรปรวนของข้อมูล ไม่ใช่ความแตกต่างที่แท้จริงของโมเดล จึงสรุปได้ว่าทั้ง 4 โมเดลมีประสิทธิภาพในระดับที่ไม่แตกต่างกันอย่างมีนัยสำคัญ การเลือกโมเดลจึงควรพิจารณาจากปัจจัยอื่น เช่น ความเสถียร ความเร็วในการประมวลผล หรือขนาดของโมเดลแทน
- การประเมินภาพรวมทุกคลาส: เมื่อพิจารณาผลการทดลองโดยรวมทั้งในแง่ประสิทธิภาพและความเหมาะสมกับการนำไปใช้งานจริง สามารถสรุปได้ว่า MobileNet-V3-Large ถือเป็นโมเดลที่เหมาะสมที่สุดสำหรับงานนี้ เนื่องจากให้ค่า F1-score สูงสุดที่ 0.8386 พร้อมกับค่า SD ต่ำสุดที่ 0.0275 แสดงให้เห็นว่ามีทั้งประสิทธิภาพและความเสถียรสูงที่สุดในกลุ่ม นอกจากนี้ยังมีจำนวน Parameter น้อยที่สุดที่ประมาณ 5.5 ล้านตัว ทำให้ประมวลผลได้รวดเร็วและสามารถ Deploy บนอุปกรณ์ที่มีทรัพยากรจำกัดได้ เช่น Mobile หรือ Edge Devices ซึ่งเหมาะสมกับบริบทการใช้งานจริงที่ต้องการระบบจำแนกรถที่เบาและรวดเร็ว GradCAM ยังยืนยันว่า MobileNet-V3-Large โฟกัสที่ feature ของตัวรถได้อย่างเฉพาะเจาะจง ในขณะที่ DenseNet-121 ถือเป็นโมเดลทำได้แย่ที่สุดในทุกด้าน ทั้ง F1-score ต่ำสุดที่ 0.7947 และ SD สูงสุดที่ 0.0591 บ่งชี้ถึงทั้งประสิทธิภาพต่ำและความไม่เสถียรสูง โดยคาดว่าปัญหามาจาก ใน Stage 1 DenseNet-121 มี trainable parameters เพียง 3,075 ตัวซึ่งน้อยกว่าโมเดลอื่นอย่างมาก ส่งผลให้ Classifier Head ยังไม่ Stable เพียงพอก่อนเข้า Stage 2 และจุดเด่นด้าน Dense Connection ที่ช่วย reuse features ได้ดีมากใน Dataset ขนาดใหญ่ แต่กลับเป็นข้อเสียในงานนี้ที่มี Dataset ขนาดเล็กเพียง 300 รูป แนวทางแก้ไขคือ Tune Hyperparameter ใหม่และเพิ่มจำนวน Dataset
- การประเมินแยกรายคลาส:
 - **Champ:** ทุก Model ให้ผลการจำแนกคลาสได้อย่างสมบูรณ์แบบ โดยมีค่า F1-score, Precision และ Recall เท่ากับ 1.0000 ± 0.0000 ในทุกโมเดลซึ่งสอดคล้องกับ Confusion Matrix ที่ไม่พบ False Positive หรือ False Negative ของคลาสนี้เลย และได้รับการยืนยันจาก GradCAM หลัง Fine-tune ที่แสดงให้เห็นว่าทุกโมเดลสามารถโฟกัสไปที่ลักษณะเฉพาะของ Champ ได้อย่างชัดเจน แสดงให้เห็นว่า Champ มีลักษณะทางภาพที่แตกต่างจากอีกสองคลาสอย่างเด่นชัด ทำให้โมเดลสามารถเรียนรู้และจำแนกได้อย่างแม่นยำ
 - **Revo:** เป็นคลาสที่ทุกโมเดลทำได้ยากที่สุด โดยมีค่า F1-score ต่ำที่สุด และจาก Confusion Matrix พบว่าข้อผิดพลาดทั้งหมดของ Revo เกิดจากการทำนายผิดเป็น Vigo เท่านั้น โดยไม่มีการสับสนกับ Champ เลย บ่งชี้ว่า Revo และ Vigo มีลักษณะทางภาพที่ซ้อนทับกัน นอกจากนี้ยังสังเกตได้ว่า Precision สูงกว่า Recall ในทุกโมเดล แสดงให้เห็นว่าเมื่อโมเดลทำนายว่าเป็น Revo มักจะถูกต้อง แต่มักพลาดรูปของ Revo บางส่วนไปทำนายเป็น Vigo แทน ซึ่งสอดคล้องกับ

GradCAM ที่แสดงให้เห็นว่าโมเดลยังไม่สามารถโฟกัสไปที่ลักษณะเฉพาะที่แยก Revo ออกจาก Vigo ได้อย่างชัดเจนพอ แนวทางแก้ไข ควรเพิ่มข้อมูล Revo ใน Training Set

- **Vigo:** มีค่า F1-score สูงกว่า Revo แต่ยังต่ำกว่า Champ และจาก Confusion Matrix พบว่า Vigo ถูกทำนายผิดเป็น Revo ในอัตราที่สูงกว่าทิศทางกลับกัน นอกจากนี้ยังสังเกตได้ว่า Precision ต่ำกว่า Recall ในทุกโมเดล ซึ่งตรงข้ามกับ Revo หมายความว่าโมเดลมักทำนายรูปของคลาสอื่นเป็น Vigo มากเกินไป โดยเฉพาะ DenseNet-121 ที่มี Precision ต่ำสุดที่ 0.6000 แนวทางแก้ไข ปรับ Class Weight ให้ Vigo มีน้ำหนักสูงขึ้น เพื่อลด Bias ที่โมเดลมีต่อ Revo, เพิ่ม Focal Loss แทน CrossEntropyLoss เพื่อให้โมเดลให้ความสนใจกับรูปที่จำแนกยากมากขึ้น โดยเฉพาะรูปที่อยู่บริเวณ Decision Boundary ระหว่าง Revo และ Vigo
- Eyeball Analysis
 - จากการวิเคราะห์ตัวอย่างรูปภาพที่โมเดลจำแนกผิดพลาดพบว่าทุกโมเดลมีรูปแบบของข้อผิดพลาดที่คล้ายคลึงกัน โดยข้อผิดพลาดทั้งหมดเกิดขึ้นระหว่างคลาส Revo และ Vigo เท่านั้น ไม่มีการสับสนกับ Champ เลย สอดคล้องกับผลจาก Confusion Matrix
 - เมื่อพิจารณารูปที่ทำนายผิดพลาดว่ามีลักษณะร่วมกัน 2 แบบคือ 1. รูปส่วนใหญ่ถ่ายจากมุมด้านข้างหรือท้ายรถ ที่ไม่สามารถมองเห็นกระจังหน้าและไฟหน้าได้ชัดเจน ซึ่งเป็น Discriminative Features หลักที่แยก Revo ออกจาก Vigo ทำให้โมเดลขาดข้อมูลสำคัญในการตัดสินใจ 2. พบรูปที่ทำนายผิดซ้ำกันในหลาย Model
 - แนวทางแก้ไข คือ Hard Example Mining การนำรูปที่ทุกโมเดลทำนายผิดซ้ำกัน มาเพิ่มใน Training Set โดยเฉพาะ การมีรูปลักษณะคล้ายกันนี้มากขึ้นจะช่วยให้โมเดลเรียนรู้ได้ดีขึ้น และการเพิ่มรูปที่ถ่ายจากมุมด้านข้างและท้ายรถของทั้งสองรุ่นให้มากขึ้นใน Training Set เพื่อให้โมเดลเรียนรู้ feature ที่แยกแยะได้แม้ในมุมที่ไม่เห็นกระจังหน้า



✖ Misclassified: 7 / 30 (23.3% error rate)

Top confused pairs (Actual → Predicted):

Vigo → Revo : 4 times
Revo → Vigo : 3 times

Error Analysis — VGG-16: Misclassified Samples



✖ Misclassified: 4 / 30 (13.3% error rate)

Top confused pairs (Actual → Predicted):

Vigo → Revo : 3 times
Revo → Vigo : 1 times

Error Analysis — ResNet-50: Misclassified Samples



✖ Misclassified: 5 / 30 (16.7% error rate)

Top confused pairs (Actual → Predicted):

Vigo → Revo : 3 times
Revo → Vigo : 2 times

Error Analysis — mobilenet_v3_large: Misclassified Samples

